NCAA Crystal Ball

Milestones 14 and 15

Question and Objective

Our main research question is: Which statistics are associated with winning the NCAA championship, and how can these findings help us build and model to predict that

Milestone 14: Data and logistic regression assumptions

Data Source

So, we are using a historical dataset for the Macfax database. We exported the stats we needed and results from PostgeSQL into a CSV file that we can use for analysis. This data has 7311 teams across 21 seasons from 2005 to 2026, excluding 202 because the NCAA tournament was canceled. Each team has the stats available from before the tournament and we have an outcome indicating whether that team won the national championship.

Load the data and inspect it

```
import pandas as pd

# Load our NCAA champion checklist dataset
data = pd.read_csv("champion_checklist.csv")

# Check the size of our dataset
print(f"Observations: {len(data):,}")
print(f"Variables: {len(data.columns)}")

# Preview some important variables
```

```
data[
    ["season", "team", "adj_em", "win_pct", "tournament_seed", "champion"]
].head()
```

```
Observations: 7,311
Variables: 45
```

	season	team	adj_em	win_pct	tournament_seed	champion
0	2005	Air Force	11.8764	0.586207	NaN	0
1	2005	Akron	7.8210	0.642857	NaN	0
2	2005	Alabama	20.1539	0.733333	5.0	0
3	2005	Alabama A&M	-7.5975	0.571429	16.0	0
4	2005	Alabama St.	-13.7944	0.517241	NaN	0

Check champ outcomes and tournament teams

```
# Keep only teams that made the NCAA tournament
tournament = data[data["tournament_seed"].notna()].copy()

# Check our tournament dataset
print(f"Tournament observations: {len(tournament):,}")
print(f"Seasons: {tournament['season'].nunique()}")
print(f"Champions: {tournament['champion'].sum()}")

# Check for duplicate team-season observations
duplicates = tournament.duplicated(
    subset=["season", "team_id"]
).sum()

print(f"Duplicate team-seasons: {duplicates}")

# Check the number of teams and champions in each season
season_summary = tournament.groupby("season").agg(
    teams=("team_id", "count"),
    champions=("champion", "sum")
)

season_summary
```

```
Tournament observations: 1,343
Seasons: 21
Champions: 21
Duplicate team-seasons: 0
```

season	teams	champions
2005	64	1
2006	64	1
2007	64	1
2008	65	1
2009	64	1
2010	65	1
2011	68	1
2012	68	1
2013	68	1
2014	61	1
2015	62	1
2016	60	1
2017	62	1
2018	63	1
2019	62	1
2021	65	1
2022	67	1
2023	61	1
2024	63	1
2025	65	1
2026	62	1

So we filtered the data to only the tournament teams, we have 1343 teams over the 21 seasons. Every season has one champion and there are no duplicate seasons or teams.

Missing values and data quality

```
# Potential predictors from the MacFax Champion Checklist
predictors = [
    "adj_em",
    "adj_o",
    "adj_d",
```

```
    "adj_tempo",
    "win_pct",
    "tournament_seed",
    "adj_efg_margin",
    "adj_tov_edge",
    "adj_reb_edge",
    "adj_ftr_margin",
    "ffi_adj",
    "wab",
    "sos_rank",
    "ap_poll_week6",
    "three_pt_pct",
    "ft_pct"
]

# Count missing values in each potential predictor
missing = tournament[predictors].isna().sum().to_frame("missing")

# Calculate the percentage of missing values
missing["percent_missing"] = (
    missing["missing"] / len(tournament) * 100
).round(2)

# Show variables with the most missing values first
missing.sort_values("percent_missing", ascending=False)
```

	missing	percent_missing
ap_poll_week6	880	65.52
adj_em	0	0.00
adj_d	0	0.00
adj_o	0	0.00
win_pct	0	0.00
tournament_seed	0	0.00
adj_efg_margin	0	0.00
adj_tempo	0	0.00
adj_tov_edge	0	0.00
adj_reb_edge	0	0.00
ffi_adj	0	0.00
adj_ftr_margin	0	0.00
wab	0	0.00
sos_rank	0	0.00

	missing	percent_missing
three_pt_pct	0	0.00
ft_pct	0	0.00

Almost all of our data is completely clean. The only thing that isn't is AP Poll week 6. But, how this works is there are only values for teams that are in the AP Poll in Week 6 every other team is Null so this is expected

Logistic Regression Assumptions

Make sure our response is binary

```
# Count champions and non-champions
class_counts = tournament["champion"].value_counts().sort_index()

# Calculate the percentage of each outcome
class_percentages = (
    tournament["champion"].value_counts(normalize=True).sort_index() * 100
)

# Display our results
outcome_summary = pd.DataFrame({
    "Count": class_counts,
    "Percentage": class_percentages.round(2)
})

outcome_summary.index = ["Non-Champion", "Champion"]

outcome_summary
```

	Count	Percentage
Non-Champion	1322	98.44
Champion	21	1.56

Our response is binary: 1 representing a national champion and 0 representing a non-chmpion, so thats making logistic regression good for this. But, our dataset is really imbalanced, only 21 out of 1343 teams are champions. This doesn't violate the assumptions, it just means we need to be careful about too many predictors and overfitting the model

Independence

```
# Count how many unique programs are in our dataset
unique_teams = tournament["team_id"].nunique()

# Count the number of seasons each program appears in
team_seasons = tournament.groupby("team_id")["season"].nunique()

# Identify programs appearing in multiple seasons
repeated_teams = (team_seasons > 1).sum()

print(f"Unique teams: {unique_teams}")
print(f"Teams appearing in multiple seasons: {repeated_teams}")
print(f"Maximum seasons for one team: {team_seasons.max()}")

# Verify that every season has exactly one champion
one_champion_per_season = (
    tournament.groupby("season")["champion"].sum() == 1
).all()

print(f"Exactly one champion per season: {one_champion_per_season}")
```

```
Unique teams: 258
Teams appearing in multiple seasons: 198
Maximum seasons for one team: 21
Exactly one champion per season: True
```

Teams aren't completely independent because they compete for the same championship each season, so only one team can win. We can still use logistic regression for this, but we need to consider this dependence when we interpret our results

Linearity

```
import statsmodels.api as sm
from scipy.stats import chi2

# Select our outcome and first predictor
model_data = tournament[["champion", "adj_em"]].dropna().copy()
```

```
# Center and scale AdjEM for easier interpretation
model_data["adj_em_5"] = (
    model_data["adj_em"] - model_data["adj_em"].mean()
) / 5

# Create a squared term to test for curvature
model_data["adj_em_squared"] = model_data["adj_em_5"] ** 2

# Model 1: Linear relationship on the log-odds scale
X_linear = sm.add_constant(model_data[["adj_em_5"]])

linear_model = sm.GLM(
    model_data["champion"],
    X_linear,
    family=sm.families.Binomial()
).fit()

# Model 2: Allow a curved relationship
X_quadratic = sm.add_constant(
    model_data[["adj_em_5", "adj_em_squared"]]
)

quadratic_model = sm.GLM(
    model_data["champion"],
    X_quadratic,
    family=sm.families.Binomial()
).fit()

# Compare the two models with a likelihood-ratio test
lr_stat = 2 * (quadratic_model.llf - linear_model.llf)
p_value = chi2.sf(lr_stat, df=1)

print(f"Linear model AIC: {linear_model.aic:.3f}")
print(f"Quadratic model AIC: {quadratic_model.aic:.3f}")
print(f"Likelihood-ratio statistic: {lr_stat:.3f}")
print(f"P-value: {p_value:.4f}")
```

```
Linear model AIC: 149.882
Quadratic model AIC: 148.735
Likelihood-ratio statistic: 3.147
P-value: 0.0761
```

We tested linearity by comparing a logistic regression using AdjEM with another model that included AdjEM sqaured. The likelihood-ratio test produced a p-value of 0.0761, which is greater than 0.05. This suggests that adding the quadratic term did not significantly improve our model. The quadratic model did have a slightly lower AIC, but the difference was small. Based on these results, we decided to continue using the linear specification for our initial regression, while recognizing that the small number of champions limits our ability to detect nonlinearity.

```
import numpy as np
import matplotlib.pyplot as plt

# Create a range of observed AdjEM values
adj_values = np.linspace(
    model_data["adj_em"].min(),
    model_data["adj_em"].max(),
    300
)

# Apply the same scaling we used when fitting the models
adj_scaled = (adj_values - model_data["adj_em"].mean()) / 5

# Predictor values for the linear model
plot_linear = pd.DataFrame({
    "const": 1.0,
    "adj_em_5": adj_scaled
})

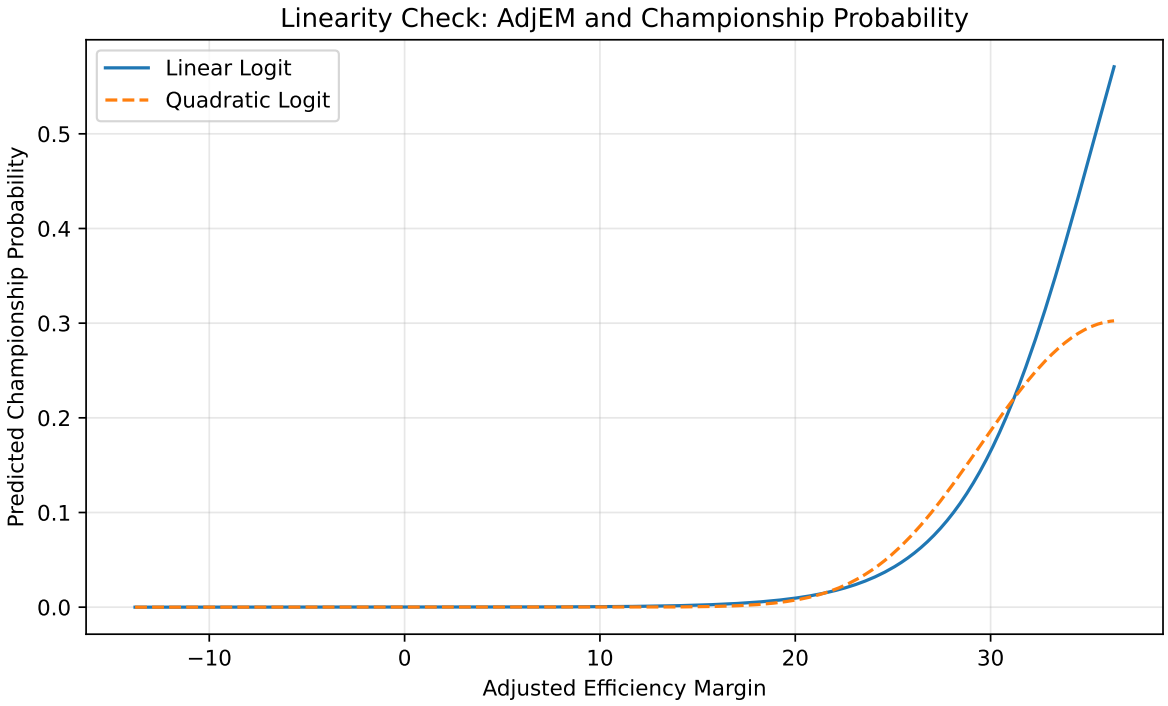
# Predictor values for the quadratic model
plot_quadratic = pd.DataFrame({
    "const": 1.0,
    "adj_em_5": adj_scaled,
    "adj_em_squared": adj_scaled ** 2
})

# Plot the predicted championship probabilities
plt.figure(figsize=(9, 5))

plt.plot(
    adj_values,
    linear_model.predict(plot_linear),
    label="Linear Logit"
)
```

```
plt.plot(
    adj_values,
    quadratic_model.predict(plot_quadratic),
    label="Quadratic Logit",
    linestyle="--"
)

plt.xlabel("Adjusted Efficiency Margin")
plt.ylabel("Predicted Championship Probability")
plt.title("Linearity Check: AdjEM and Championship Probability")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



Identifiability and Multicolinearity

```
# Select potential predictors from our checklist
candidate_predictors = [
    "adj_em",
```

```
    "adj_o",
    "adj_d",
    "win_pct",
    "tournament_seed",
    "ffi_adj",
    "wab"
]

# Calculate correlations between predictors
correlations = tournament[candidate_predictors].corr()

correlations.round(2)
```

	adj_em	adj_o	adj_d	win_pct	tournament_seed	ffi_adj	wab
adj_em	1.00	0.81	-0.68	0.57	-0.88	0.98	0.89
adj_o	0.81	1.00	-0.12	0.44	-0.69	0.77	0.68
adj_d	-0.68	-0.12	1.00	-0.42	0.62	-0.71	-0.66
win_pct	0.57	0.44	-0.42	1.00	-0.50	0.54	0.71
tournament_seed	-0.88	-0.69	0.62	-0.50	1.00	-0.88	-0.86
ffi_adj	0.98	0.77	-0.71	0.54	-0.88	1.00	0.88
wab	0.89	0.68	-0.66	0.71	-0.86	0.88	1.00

After looking at the correlations between our potenital predictors, we have a bynch of strong relationships. AdjEM and FFI have a correlation of 0.98, also AdjEM has strong relationships with WAB and tournament seed. There might be serveral predictors with overlapping information. Including them straight up together could create multicolinearity problems and make it difficult to interpret their individual effects. So, we will be careful about which variables we include together in our regression models

Check multicolinearity

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm

# Select predictors with potentially overlapping information
vif_predictors = [
    "adj_em",
    "win_pct",
    "tournament_seed",
```

```
    "ffi_adj",
    "wab"
]

# Add the intercept
X_vif = sm.add_constant(tournament[vif_predictors])

# Calculate VIF for each predictor
vif_results = pd.DataFrame({
    "Variable": X_vif.columns,
    "VIF": [
        variance_inflation_factor(X_vif.values, i)
        for i in range(X_vif.shape[1])
    ]
})

# Exclude the intercept
vif_results = vif_results[
    vif_results["Variable"] != "const"
]

vif_results.sort_values("VIF", ascending=False).round(2)
```

	Variable	VIF
1	adj_em	34.31
4	ffi_adj	31.55
5	wab	8.57
3	tournament_seed	5.58
2	win_pct	2.33

Reduce predictors

```
# Choose two predictors for a simpler model
reduced_predictors = [
    "adj_em",
    "win_pct"
]

# Add an intercept
```

```
X_reduced = sm.add_constant(tournament[reduced_predictors])

# Calculate VIF again
reduced_vif = pd.DataFrame({
    "Variable": X_reduced.columns,
    "VIF": [
        variance_inflation_factor(X_reduced.values, i)
        for i in range(X_reduced.shape[1])
    ]
})

# Remove intercept and display results
reduced_vif = reduced_vif[
    reduced_vif["Variable"] != "const"
]

reduced_vif.round(2)
```

	Variable	VIF
1	adj_em	1.48
2	win_pct	1.48

Validity and Representativeness

```
# Count tournament teams and champions in each season
coverage = tournament.groupby("season").agg(
    recorded_teams=("team_id", "nunique"),
    champions=("champion", "sum")
)

# Expected NCAA tournament field sizes
# 65 teams through 2010, 68 teams beginning in 2011
coverage["expected_teams"] = coverage.index.map(
    lambda year: 65 if year <= 2010 else 68
)

# Calculate how many tournament teams are missing
coverage["missing_teams"] = (
    coverage["expected_teams"] - coverage["recorded_teams"]
```

```
)

# Calculate the percentage of each field represented
coverage["coverage_pct"] = (
    coverage["recorded_teams"] / coverage["expected_teams"] * 100
).round(1)

coverage
```

season	recorded_teams	champions	expected_teams	missing_teams	coverage_pct
2005	64	1	65	1	98.5
2006	64	1	65	1	98.5
2007	64	1	65	1	98.5
2008	65	1	65	0	100.0
2009	64	1	65	1	98.5
2010	65	1	65	0	100.0
2011	68	1	68	0	100.0
2012	68	1	68	0	100.0
2013	68	1	68	0	100.0
2014	61	1	68	7	89.7
2015	62	1	68	6	91.2
2016	60	1	68	8	88.2
2017	62	1	68	6	91.2
2018	63	1	68	5	92.6
2019	62	1	68	6	91.2
2021	65	1	68	3	95.6
2022	67	1	68	1	98.5
2023	61	1	68	7	89.7
2024	63	1	68	5	92.6
2025	65	1	68	3	95.6
2026	62	1	68	6	91.2

Constant Variance and Normality

Constant variance and normally distributed errors are not required assumptions for our logistic regression model. Since our response variable is binary, it follows a Bernoulli distribution, where the variance depends on the predicted probability. Because of this, we do not expect constant residual variance or normally distributed residuals, and we do not treat these as violations of our model.